

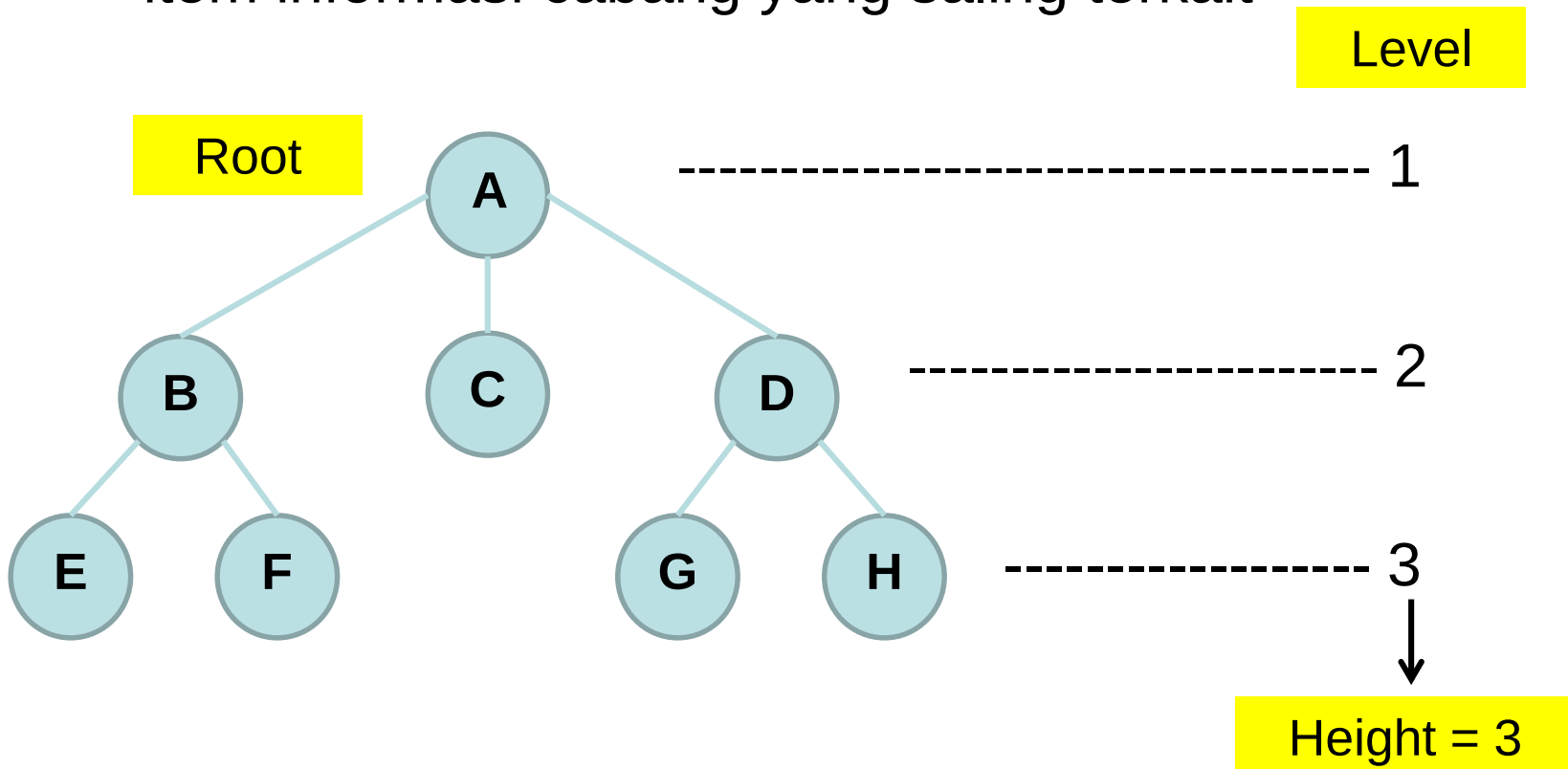
Pertemuan 6



STRUKTUR POHON & KUNJUNGAN POHON BINER

DEFINISI POHON (TREE)

- Pohon (Tree) termasuk struktur non linear yang didefinisikan sebagai data yang terorganisir dari suatu item informasi cabang yang saling terkait



Istilah – istilah Dalam Pohon

1. Predesesor

Node yang berada diatas node tertentu.
(contoh : B predesesor dari E dan F)

2. Succesor

Node yang berada dibawah node tertentu.
(contoh : E dan F merupakan succesor dari B)

3. Ancestor

Seluruh node yang terletak sebelum node tertentu dan terletak pada jalur yang sama.
(contoh : A dan B merupakan ancestor dari F)

4. **Descendant**

Seluruh node yang terletak sesudah node tertentu dan terletak pada jalur yang sama.

(contoh : F dan B merupakan ancestor dari A)

5. **Parent**

Predecessor satu level diatas satu node

(contoh : B merupakan parent dari F)

6. **Child**

Succesor satu level dibawah satu node

(contoh : F merupakan child dari B)

7. **Sibling**

Node yang memiliki parent yang sama dengan satu node (contoh : E dan F adalah sibling)

8. Subtree

Bagian dari tree yang berupa suatu node beserta descendant-nya (contoh : Subtree B, E, F dan Subtree D, G, H)

9. Size

Banyaknya node dalam suatu tree (contoh : gambar tree diatas memiliki size = 8)

10. Height

Banyaknya tingkat/level dalam suatu tree (contoh : gambar tree diatas memiliki height = 3)

11. Root (Akar)

Node khusus dalam tree yang tidak memiliki predesesor (Contoh : A)

12. Leaf (Daun)

Node-node dalam tree yang tidak memiliki daun (contoh : Node E,F,C,G,H)

13. Degree (Derajat)

Banyaknya child yang dimiliki oleh suatu node (contoh : Node A memiliki derajat 3, node B memiliki derajat 2)

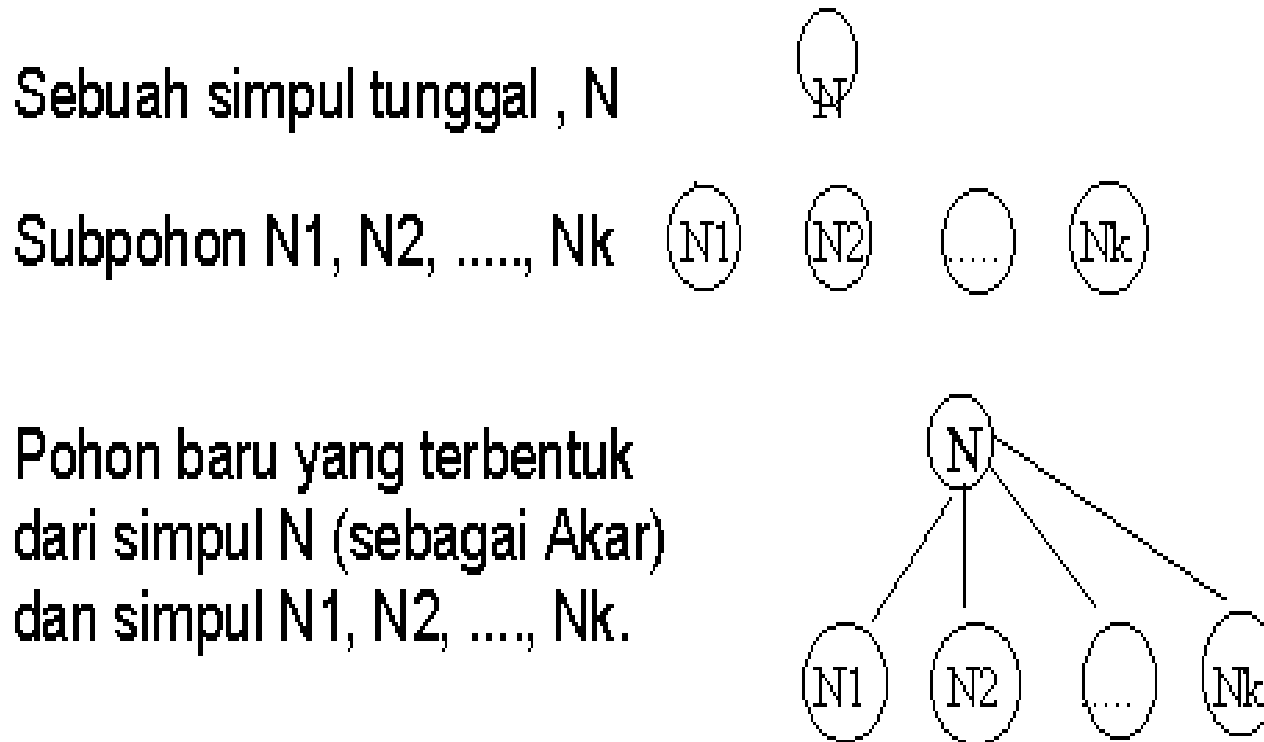
ISTILAH-ISTILAH DASAR

Pohon atau Tree adalah salah satu bentuk Graph terhubung yang tidak mengandung sirkuit.

Karena merupakan Graph terhubung, maka pada Pohon (Tree) selalu terdapat Path atau Jalur yang menghubungkan setiap simpul dalam dua pohon.

Pohon (Tree) dapat juga didefinisikan sebagai kumpulan elemen yang salah satu elemennya disebut dengan Akar (Root) dan sisa elemen lain (Simpul) yang terpecah menjadi sejumlah himpunan yang saling tidak berhubungan yang disebut dengan Subpohon (Subtree) atau cabang

Gambar berikut ini menjelaskan tentang pembentukan awal dari Pohon (Tree).



Sifat Utama Pohon Berakar

1. Jika Pohon mempunyai Simpul sebanyak n , maka banyaknya ruas atau edge adalah $(n-1)$.
2. Mempunyai Simpul Khusus yang disebut Root, jika Simpul tersebut memiliki derajat keluar ≥ 0 , dan derajat masuk = 0.
3. Mempunyai Simpul yang disebut sebagai Daun / Leaf, jika Simpul tersebut berderajat keluar = 0, dan berderajat masuk = 1.
4. Setiap Simpul mempunyai Tingkatan / Level yang dimulai dari Root yang Levelnya = 1 sampai dengan Level ke - n pada daun paling bawah. Simpul yang mempunyai Level sama disebut Bersaudara atau Brother atau Stribling.

5. Pohon mempunyai Ketinggian atau Kedalaman atau Height, yang merupakan Level tertinggi
6. Pohon mempunyai Weight atau Berat atau Bobot, yang banyaknya daun (leaf) pada Pohon.
7. Banyaknya Simpul Maksimum sampai Level N adalah :

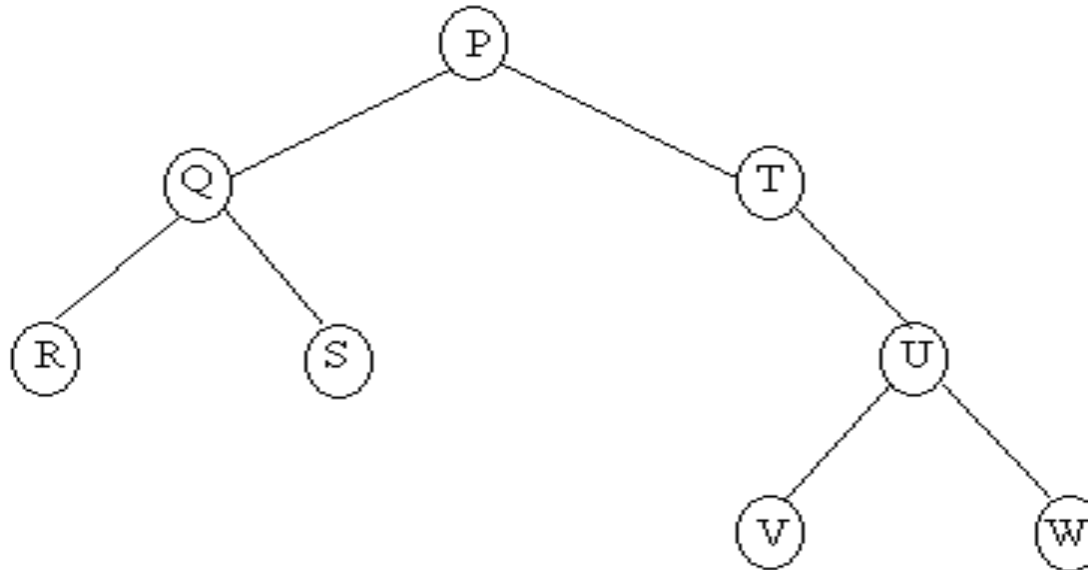
$$2^{(N)} - 1$$

8. Banyaknya Simpul untuk setiap Level I adalah :

$$\sum_{I=1}^N 2^{(I-1)}$$

Hutan (Forest) adalah kumpulan Pohon yang tidak saling berhubungan

Diketahui suatu bentuk Pohon Berakar T sebagai berikut :



Pohon Diatas Mempunyai :

- a. Simpul sebanyak = 8 dan edge = $n - 1 = 8 - 1 = 7$
- b. Root pada Pohon T diatas adalah Simpul P
- c. Mempunyai daun (Leaf) = 4, yaitu = R, S, V dan W

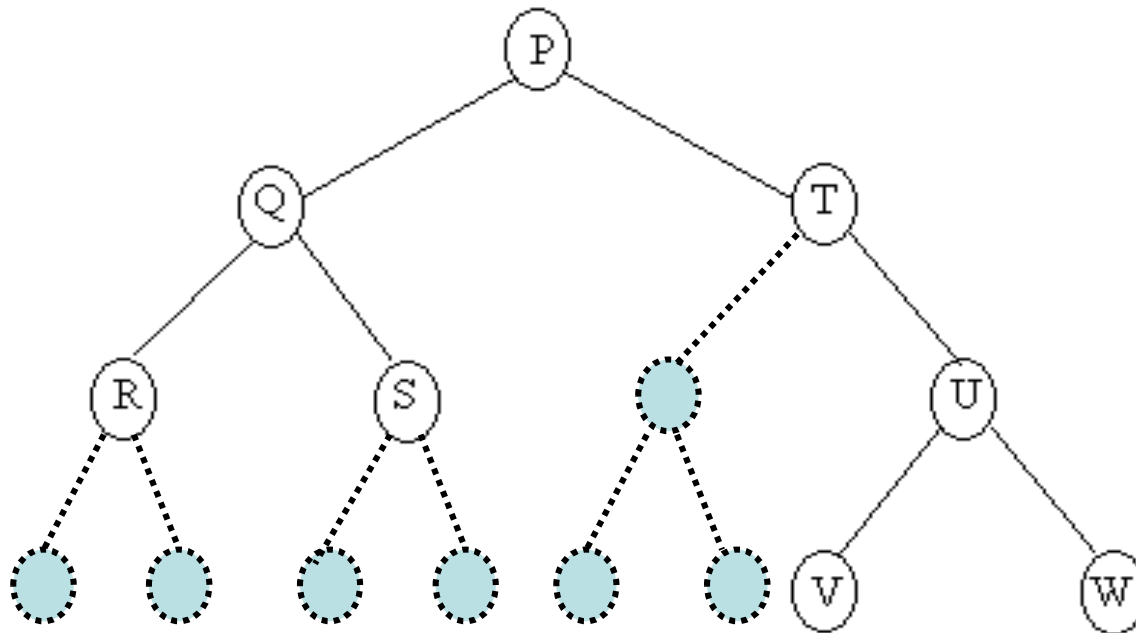
- d. Level (tingkatan) Pohon = 4 yaitu :
- Level 1 = Simpul P
 - Level 2 = Simpul Q dan T
 - Level 3 = Simpul R, S dan U
 - Level 4 = Simpul V dan W
- e. Ketinggian atau kedalaman = jumlah level = 4
- f. Weight atau berat atau bobot = jumlah daun = 4

Dalam gambar Pohon T diatas dapat dibentuk 2 buah hutan (forest), bila simpul P dihilangkan, yaitu :

Hutan 1 : Q,R,S

Hutan 2 : T,U,V,W

- g. Banyaknya Simpul Maksimum yang dapat terbentuk sampai Level 4 (bila simpul pada pohon dianggap penuh) adalah : $2^N - 1$
 $2^{(4)} - 1 = 16 - 1 = 15$

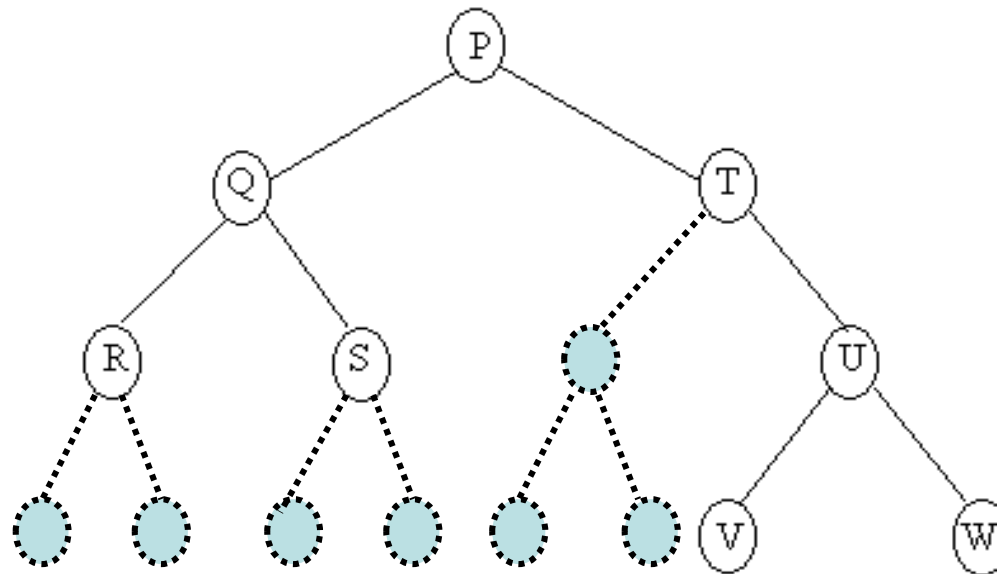


- h. Banyaknya Simpul maksimum untuk setiap Level l
(bila simpul pada pohon dianggap penuh) adalah :
Maksimum Simpul pada level 2 = $2^{(1-1)}$

$$= 2^{(2-1)} = 2$$

$$\text{Maksimum Simpul pada level 3} = 2^{(3-1)} = 4$$

Maksir



POHON BINAR (BINARY TREE)

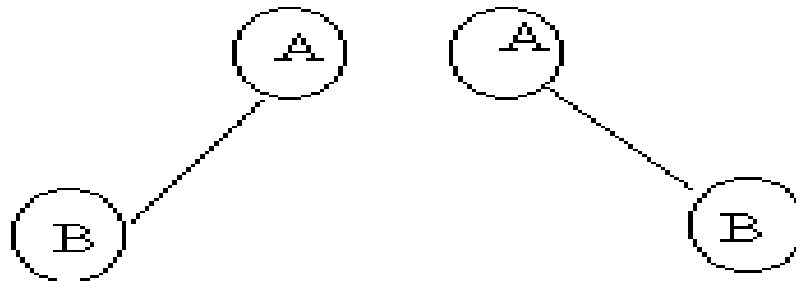
Struktur ini biasanya digunakan untuk menyajikan data yang mengandung hubungan hirarkial antara elemen-elemennya.

Bentuk Pohon Berakar yang lebih mudah dikelola dalam komputer adalah Pohon Biner (Binary Tree) yang lebih dikenal sebagai Pohon Umum (General Tree) yang dapat didefinisikan sebagai kumpulan simpul yang mungkin kosong atau mempunyai akar dan dua Subpohon yang saling terpisah yang disebut dengan Subpohon Kiri / cabang kiri (Left Subtree) dan Subpohon Kanan / cabang kanan (Right Subtree).

Karakteristik Pohon Binar (Binary Tree) :

1. Setiap Simpul paling banyak hanya memiliki dua buah anak
2. Derajat Tertinggi dari setiap Simpul adalah dua.
3. Dibedakan antara Cabang Kiri dan Cabang Kanan.
4. Dimungkinkan tidak mempunyai Simpul

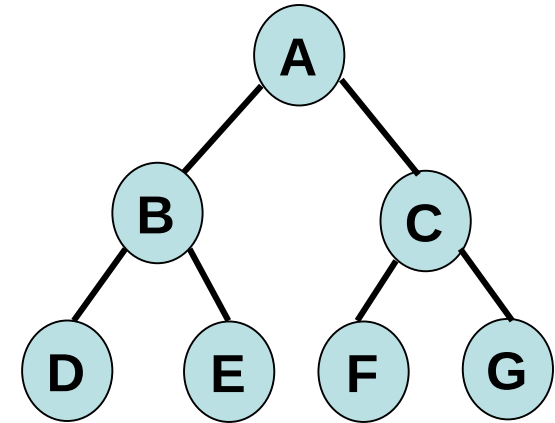
Berikut ini diberikan contoh gambar Pohon Binar (Binary Tree) dengan *Cabang Kiri* dan *Cabang Kanan*.



ISTILAH PADA POHON BINER

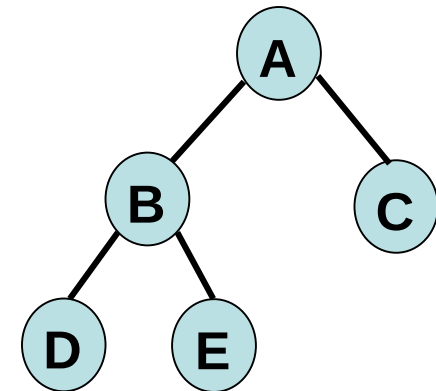
- **Pohon Biner Penuh (Full Binary Tree)**

Semua simpul (kecuali daun) memiliki 2 anak dan tiap cabang memiliki panjang ruas yang sama



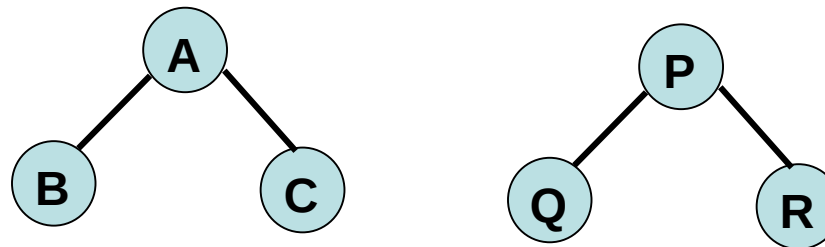
- **Pohon Biner Lengkap (Complete Binary Tree)**

Hampir sama dengan Pohon Biner Penuh, semua simpul (kecuali daun) memiliki 2 anak tetapi tiap cabang memiliki panjang ruas berbeda



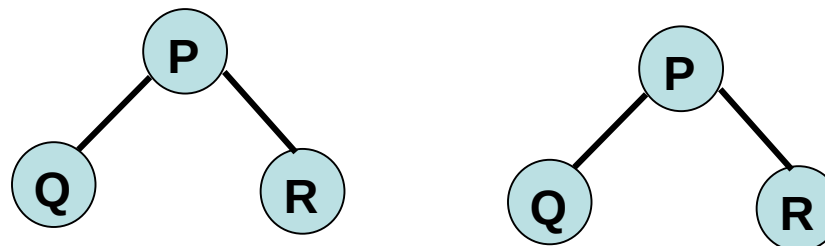
- **Pohon Biner Similer**

Dua pohon yang memiliki struktur yang sama tetapi informasinya berbeda



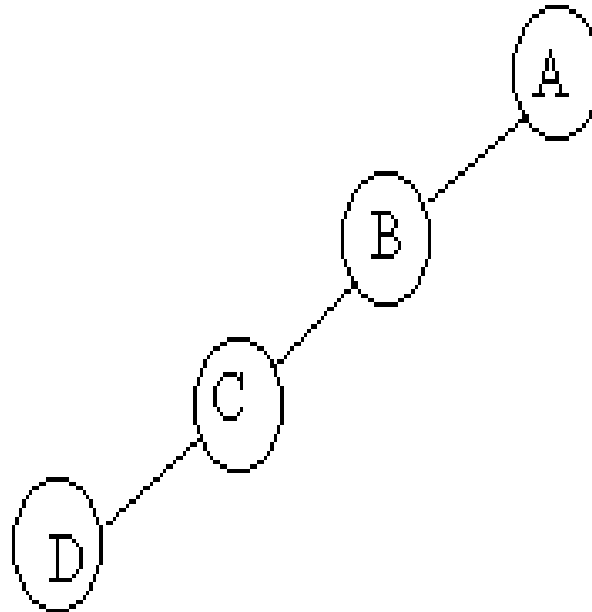
- **Pohon Biner Ekuivalent**

Dua pohon yang memiliki struktur dan informasi yang sama



- **Pohon Biner Miring (Skewed Tree)**

Dua pohon yang semua simpulnya mempunyai satu anak / turunan kecuali daun



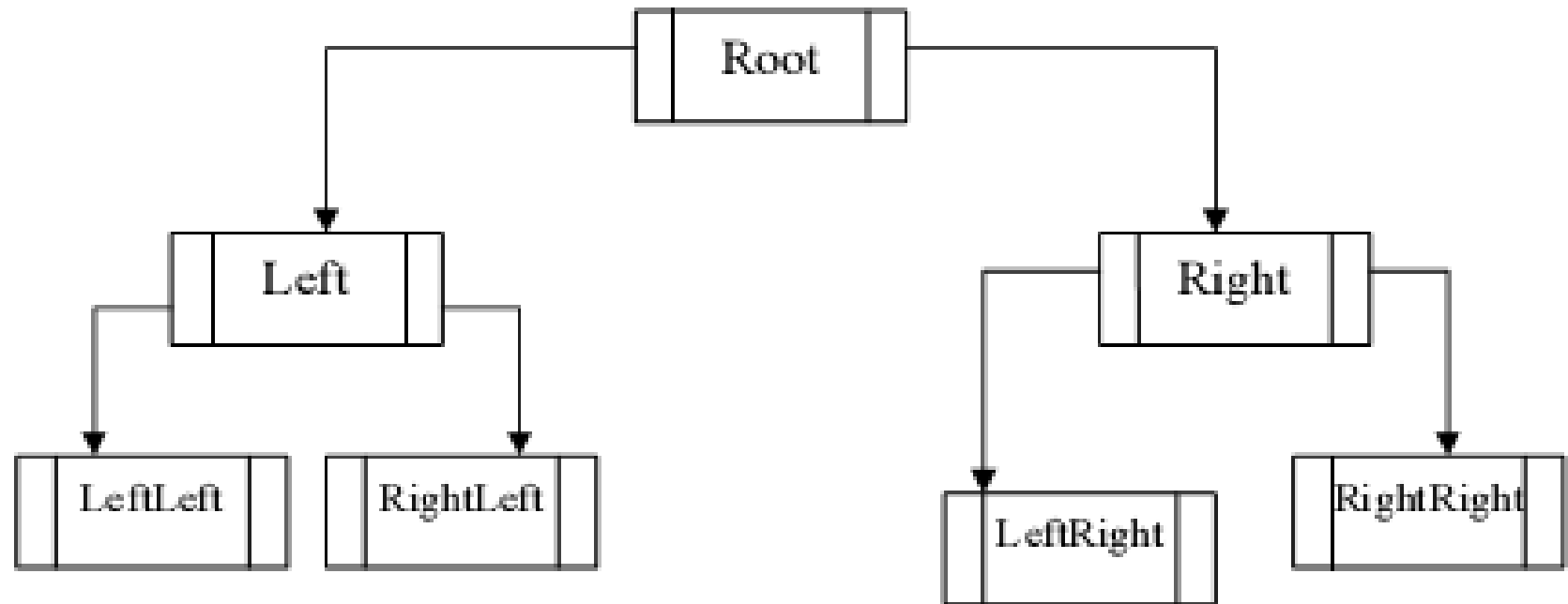
Deklarasi Pohon Biner

Dalam setiap simpul selalu berisi dua buah Pointer untuk menunjuk ke cabang Kiri dan cabang Kanan dan informasi yang akan disimpan dalam simpul tersebut.

Definisi Node

```
# The Node Class defines the structure of a Node
class Node:
# Initialize the attributes of Node
def __init__(self, data):
self.left = None # Left Child
self.right = None # Right Child
self.data = data # Node Data
```

Ilustrasi:



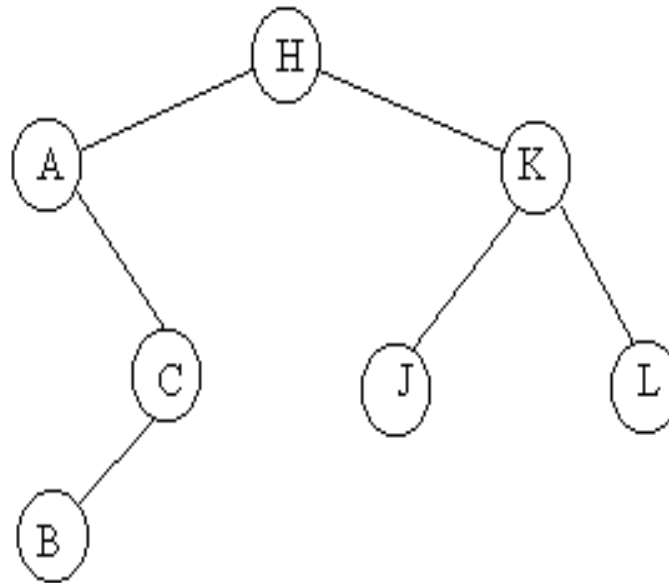
Penyajian Pohon Binar (Binary Tree)

- Tree dapat dibuat dengan menggunakan linked list secara rekursif.
- Linked list yang digunakan adalah double linked list non circular
- Data yang pertama kali masuk akan menjadi node root.
- Data yang lebih kecil dari data node root akan masuk dan menempati node kiri dari node root, sedangkan jika lebih besar dari data node root, akan masuk dan menempati node di sebelah kanan node root.

Bila diberikan untai HAKJCBL, maka proses untuk dapat membentuk pohon biner dari untai diatas adalah :

1. Karakter pertama 'H' ditempatkan sebagai akar (root)
2. Karakter 'A', karena lebih kecil dari 'H', maka akan menempati cabang kiri.
3. Karakter 'K', karena lebih besar dari 'H', maka akan menempati cabang kanan.
4. Karakter 'J', lebih besar dari 'H' dan kecil dari 'K', maka menempati cabang kiri 'K'.
5. Karakter 'C', karena lebih besar dari 'A', maka akan menempati cabang kanan.
6. Karakter 'B', karena lebih kecil dari 'C', maka akan menempati cabang kiri.
7. Karakter 'L', lebih besar dari 'K', maka menempati cabang kiri kanan.

Sehingga terbentuk pohon biner seperti berikut :

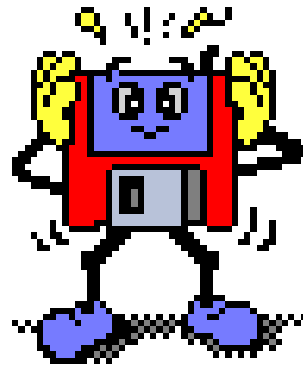


Latihan

Buatlah pohon biner dari barisan bilangan berikut:

1. 12, 22, 8, 19, 10, 9, 20, 4, 2, 6
2. 2, 3, 4, 5, 50, 10, 15, 13, 20, 12, 10, 7
3. 7, 13, 4, 6, 5, 9, 15, 20, 60, 14, 40, 70
4. 50, 45, 55, 41, 49, 13, 60, 70, 40, 35, 30, 20, 80, 75, 85
5. 12, 19, 11, 17, 29, 21, 20, 22, 13, 14, 18, 16, 15

KUNJUNGAN PADA POHON BINER



Kunjungan Pohon Biner

Kunjungan pada Pohon Binar merupakan salah satu operasi yang sering dilakukan pada suatu Pohon Binar tepat satu Kali (Binary Tree Traversal). Operasi ini terbagi menjadi 3 bentuk :

1. Kunjungan secara Preorder (Depth First Order), mempunyai urutan :
 - a. Cetak isi simpul yang dikunjungi (Simpul Akar)
 - b. Kunjungi Cabang Kiri
 - c. Kunjungi Cabang Kanan

Kunjungan Pohon Biner (Lanjutan)

2. Kunjungan secara Inorder (Symetric Order), mempunyai urutan :
 - a. Kunjungi Cabang Kiri
 - b. Cetak isi simpul yang dikunjungi (Simpul Akar)
 - c. Kunjungi Cabang Kanan

3. Kunjungan secara Postorder, mempunyai urutan :
 - a. Kunjungi Cabang Kiri
 - b. Kunjungi Cabang Kanan
 - c. Cetak isi simpul yang dikunjungi (Simpul Akar)

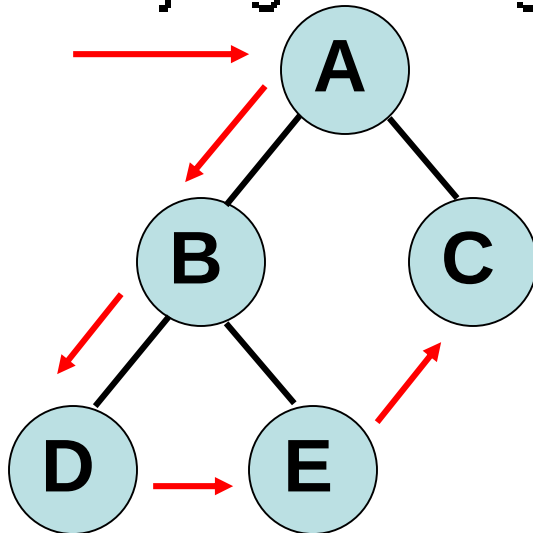
Kunjungan Pohon Biner (Lanjutan)

Pada ketiga cara kunjungan diatas, kunjungan ke Cabang Kiri dilakukan terlebih dahulu, baru kemudian kunjungan ke Cabang Kanan. Dengan orientasi semacam ini, Ketiga kunjungan diatas disebut dengan Left To Right Oriented (LRO).

Jika kunjungan ke Cabang Kanan dilakukan lebih dahulu baru kemudian kunjungan ke Cabang Kiri, maka Orientasi semacam ini disebut Right To Left Oriented (RLO).

PreOrder

1. Kunjungan secara Preorder (Depth First Order), mempunyai urutan :
 - a. Cetak isi simpul yang dikunjungi (Simpul Akar)
 - b. Kunjungi Cabang Kiri
 - c. Kunjungi Cabang Kanan

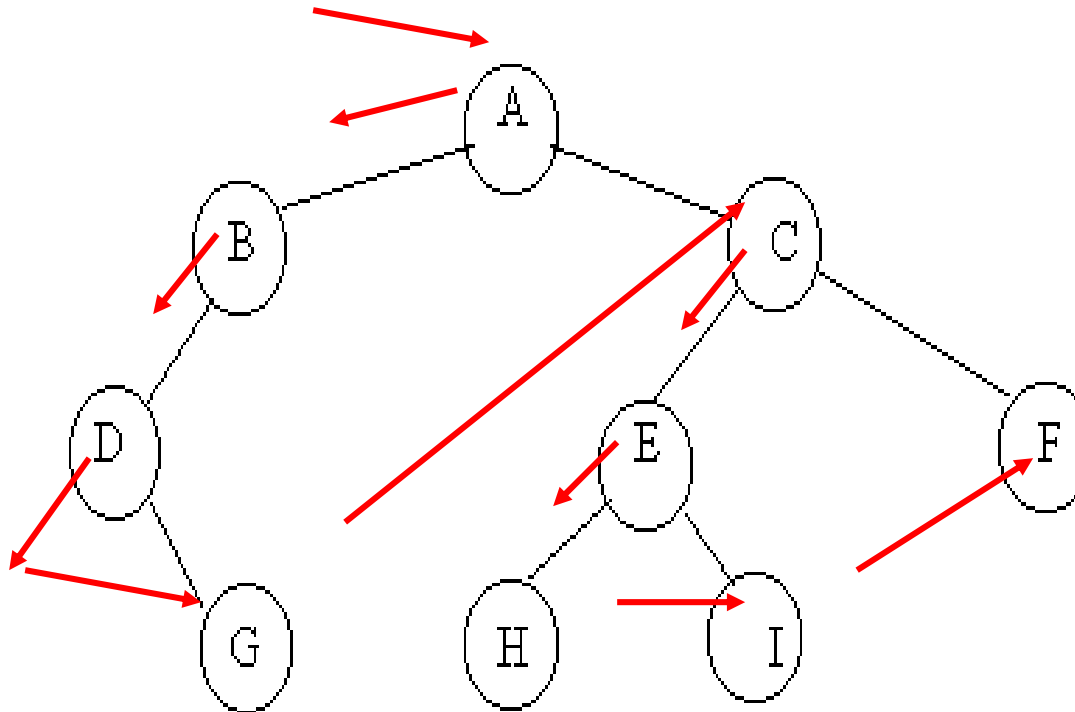


A B D E C

Klik Animasi

Contoh PreOrder

Kunjungan PreOrder



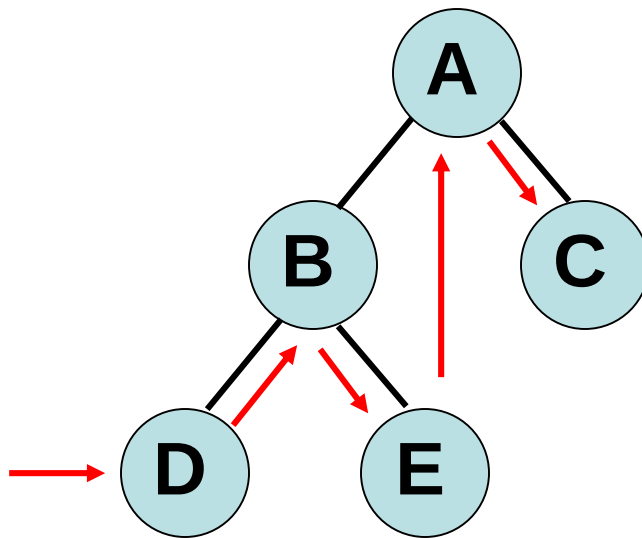
Hasil : A B D G C E H I F

Klik Animasi

InOrder

2. Kunjungan secara Inorder (Symetric Order), mempunyai urutan :

- Kunjungi Cabang Kiri
- Cetak isi simpul yang dikunjungi (Simpul Akar)
- Kunjungi Cabang Kanan

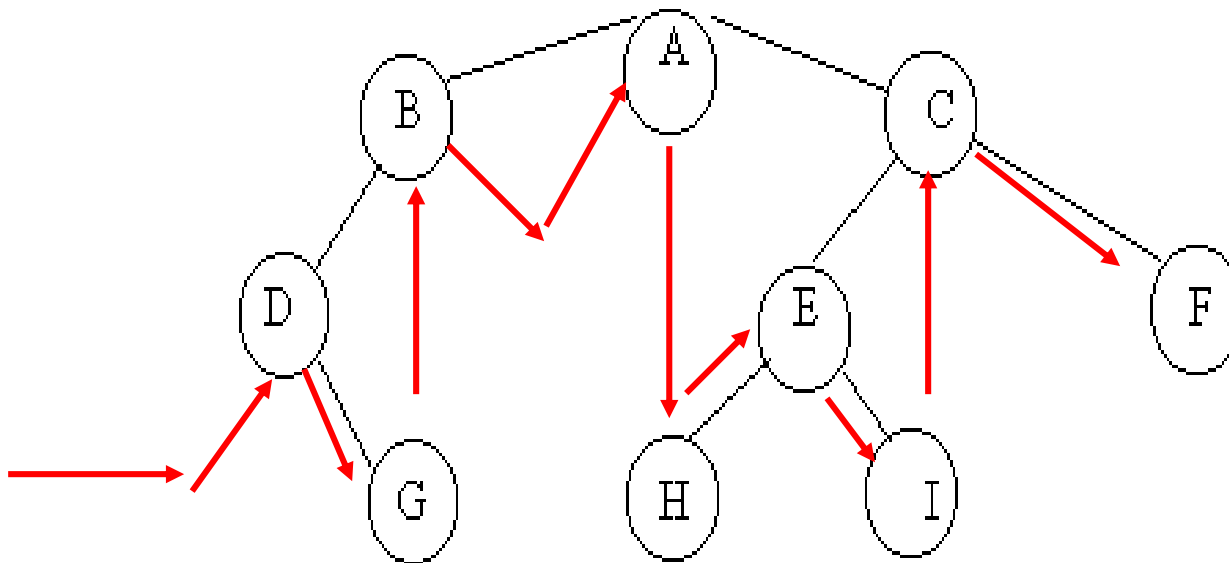


D B E A C

Klik Animasi

Contoh InOrder

Kunjungan InOrder

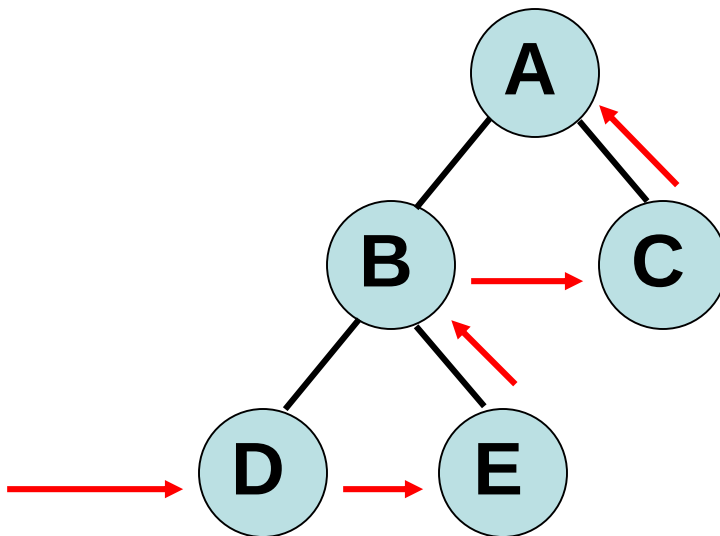


Hasil : D G B A H E I C F

Klik Animasi

PostOrder

3. Kunjungan secara Postorder, mempunyai urutan :
- a. Kunjungi Cabang Kiri
 - b. Kunjungi Cabang Kanan
 - c. Cetak isi simpul yang dikunjungi (Simpul Akar)

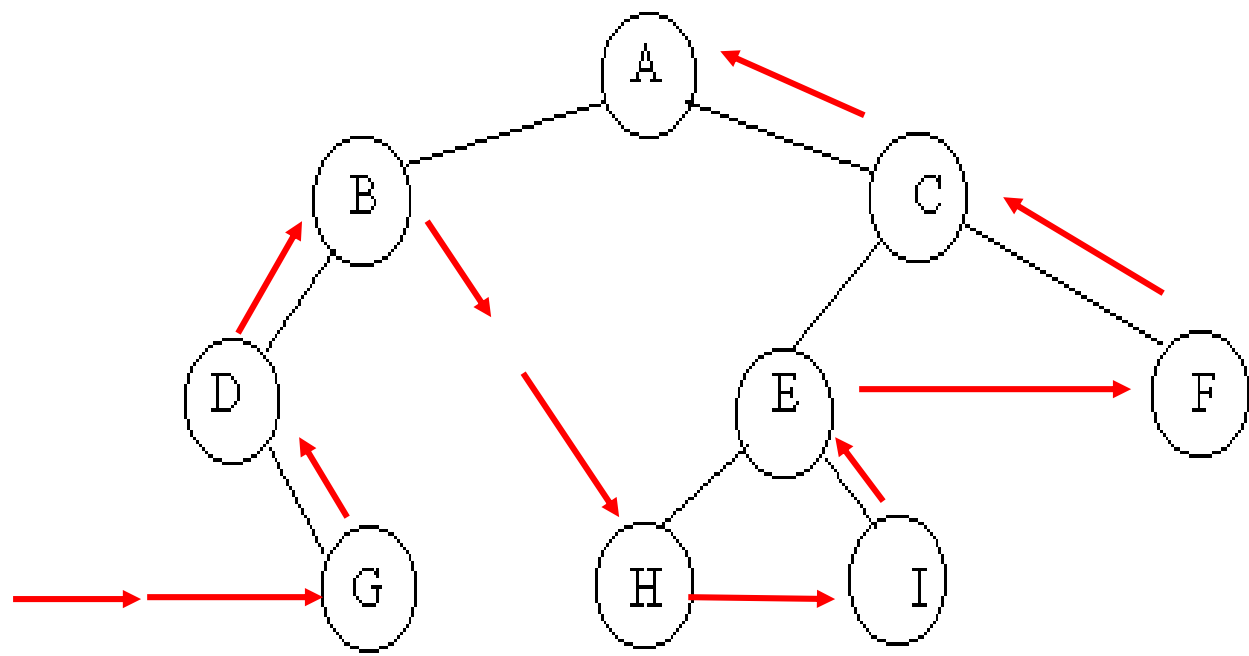


D E B C A

Klik Animasi

Contoh PostOrder

Kunjungan PostOrder



Hasil : G D B H I E F C A

Klik Animasi

Secara singkat kunjungan Level Order ini dapat dijelaskan sebagai berikut.

1. Dimulai dengan memasukkan Akar kedalam antrean.
2. Kemudian mengeluarkan Akar tersebut keluar dari antrean.

Pada saat Akar tersebut dikeluarkan dari antrean, cabang kiri dan cabang kanan secara berturut-turut dimasukkan dalam antrean.

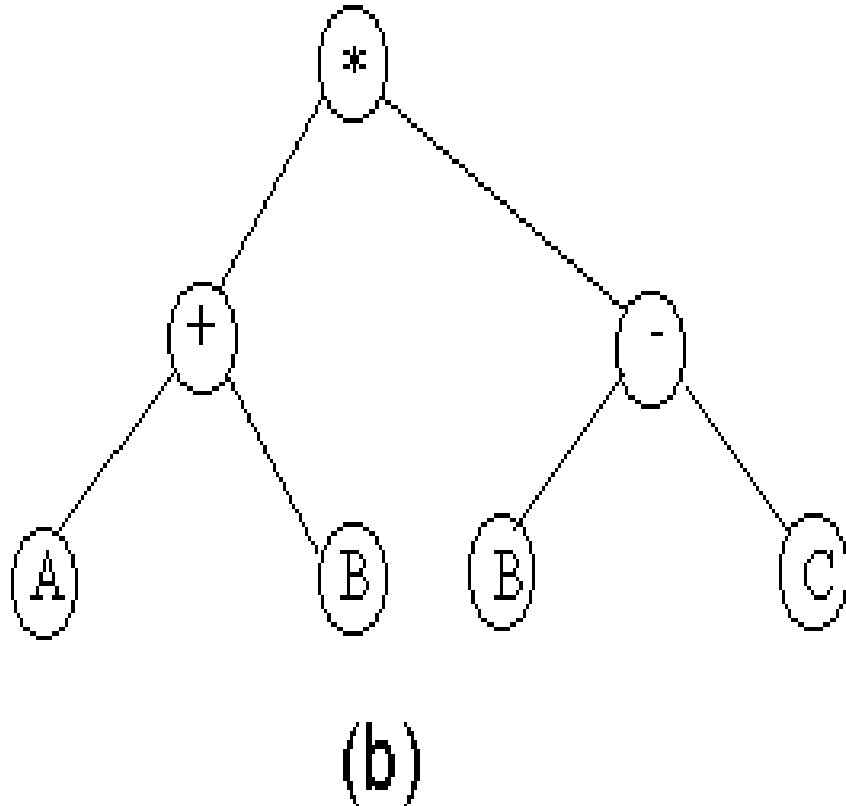
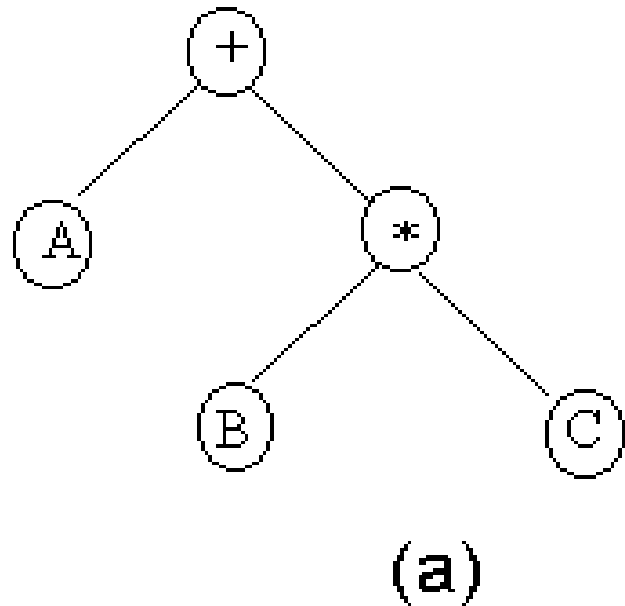
Dengan kata lain jika suatu elemen dikeluarkan dari antrean, maka cabang kiri dan kanan dari elemen yang baru saja dikeluarkan dimasukkan kedalam antrean.

APLIKASI POHON BINER

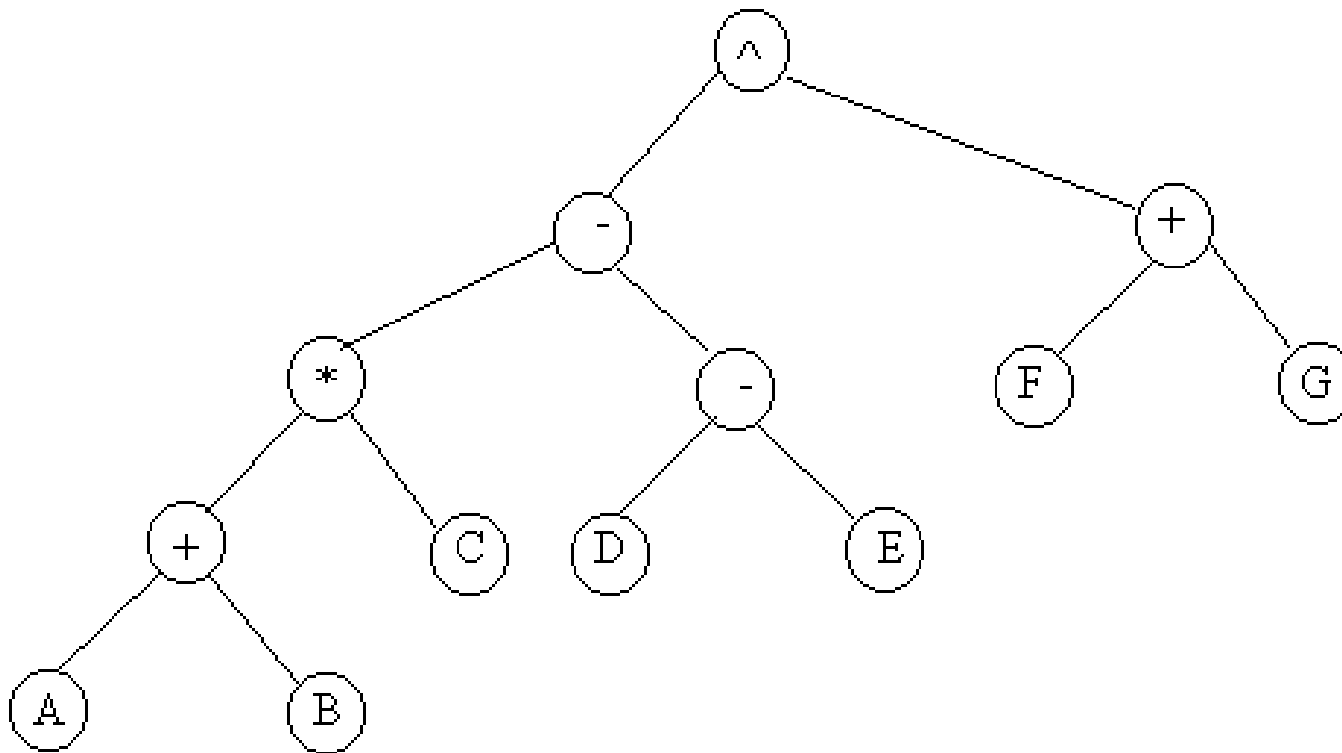
NOTASI PREFIX, INFIX DAN POSTFIX

Pada bagian ini akan dibahas tentang bagaimana menyusun sebuah Pohon Binar yang apabila dikunjungi secara PreOrder akan menghasilkan Notasi Prefix, kunjungan secara InOrder menghasilkan Notasi Infix, dan kunjungan PostOrder menghasilkan Notasi Postfix.

Aplikasi Pohon Biner (Lanjutan)



Aplikasi Pohon Biner (Lanjutan)



(c)

Aplikasi Pohon Biner (Lanjutan)

Berdasarkan Gambar diatas, apabila dilakukan kunjungan secara PreOrder, maka akan diperoleh Notasi Prefix dari persamaan-persamaan yang digambarkan tersebut, yaitu :

$+A*BC$ (Gambar.a)

$*+AB-BC$ (Gambar.b)

$^-*+ABC-DE+FG$ (Gambar.c)

Jika dilakukan kunjungan secara InOrder, akan diperoleh Notasi Infixnya, yaitu :

$(A+(B*C))$ (Gambar.a)

$((A+B) * (B-C))$ (Gambar.b)

$((((A+B) * C) - (D-E))^(F+G))$ (Gambar.c)

Aplikasi Pohon Biner (Lanjutan)

Jika dilakukan kunjungan secara PostOrder, akan diperoleh Notasi Postfixnya, yaitu :

ABC^{*+} (Gambar.a)

$AB+BC^{-*}$ (Gambar.b)

$AB+C^{*}DE--FG+^{\wedge}$ (Gambar.c)